Ego Research: Consolidated Experiment Report

Ego Research

October 9, 2026

Abstract

This report consolidates two closed-loop LIBERO reproductions carried out on a single A100 GPU during 2026.10.05–2026.10.11. Experiment A evaluates the released OpenWAM-α LIBERO policy under its native protocol and obtains $1985/2000 = 99.25\,\%$ (official $99.30\,\%$). Experiment B evaluates the released $\pi_{0.5}$ LIBERO policy with the official OpenPI implementation and obtains $1935/2000 = 96.75\,\%$ (official $96.85\,\%$). Neither model was retrained. Each experiment is written as a self-contained section pulled in from `reports/experiments/`, and the figures are shared from `figures/`. The two results answer each author's own reproduction question and must not be compared as a fair architectural comparison.

Contents

1 Experiment A: OpenWAM-α on LIBERO **2**
- 1.1 Overview and Scope . . . 2
- 1.2 The LIBERO Benchmark . . . 2
- 1.3 Model and Inference Architecture . . . 3
- 1.4 Closed-Loop Data Pipeline . . . 3
- 1.5 Frozen Environment and Provenance . . . 4
- 1.6 Results . . . 5
- 1.7 Compute, Timing and Control Budget . . . 8
- 1.8 Failure Inventory . . . 9
- 1.9 Reproducibility Notes . . . 9
- 1.10 Teaching Case: Images, Actions and Video . . . 9
- 1.11 Discussion: Reading OpenWAM Alongside $\pi_{0.5}$ . . . 12
- 1.12 Limitations . . . 13
- 1.13 Summary . . . 13

2 Experiment B: $\pi_{0.5}$ on LIBERO **13**
- 2.1 Overview and Scope . . . 13
- 2.2 Benchmark and Evaluation Protocol . . . 14
- 2.3 Model and Configuration . . . 14
- 2.4 Closed-Loop Data Pipeline . . . 15
- 2.5 Frozen Source and Environment . . . 16
- 2.6 Results . . . 16
- 2.7 Compute, Timing and Archive . . . 19
- 2.8 Teaching Cases: Six Genuine Episodes . . . 20
- 2.9 Numerical Anomalies and Non-Finite Failures . . . 22
- 2.10 Normal Failures: Open Drawer, Still a Failure . . . 22
- 2.11 Limitations and Next Steps . . . 23
- 2.12 Summary . . . 24

1 Experiment A: OpenWAM-α on LIBERO

1.1 Overview and Scope

This experiment reproduces the published LIBERO success rate of the *OpenWAM-α* policy [3] on a single A100 GPU. No new weights were trained and no per-seed tuning was performed: the authors' released LIBERO checkpoint is evaluated under its native task protocol, and the resulting score is compared against the officially reported number.

LIBERO contains 130 language-conditioned manipulation tasks in total. This evaluation covers the four standard suites — Spatial, Object, Goal and LIBERO-10 — that together contain 40 tasks. Each task is rolled out for 50 episodes, giving $40 \times 50 = 2000$ closed-loop episodes. The complete task definitions, scene assets and fixed initial states are taken from a pinned version of LIBERO. Expert demonstrations are not required for evaluation and the authors' full training set was not downloaded.

1.2 The LIBERO Benchmark

LIBERO is a language-conditioned manipulation benchmark for simulated robot arms [2]. Every task defines a tabletop scene, a natural-language goal and a BDDL goal predicate. At each control step the policy observes the current external and wrist RGB frames together with the robot's proprioceptive state, predicts an action, and the MuJoCo simulator advances the scene to produce the next observation. This is a closed loop, not video classification and not a real-robot deployment.

The raw data therefore consists of scene assets, BDDL task descriptions, language instructions and fixed initial states. RGB frames are rendered dynamically from the geometry, joint, mass, friction, contact and camera configuration of the simulator; the model is never handed a pre-recorded future video. Object ground-truth positions and the success predicate are maintained by the environment and are not extra policy inputs.

A few terms are used with a precise meaning throughout this report:

- **task** — the goal, for example *"put the milk in the basket"*;
- **trial / episode** — one complete attempt starting from a fixed initial state;
- **control step** — one execution of a 7-dimensional arm command;
- **query** — one network planning call, which may emit several actions at once.

Trial IDs 0–49 index the 50 attempts of a task and select the initial state; they are *not* seeds. A seed is the origin of a random-number stream; fixing it aids reproducibility but is not by itself sufficient — initial states, library versions, numerical precision and model stochasticity must be fixed as well.

The final LIBERO action is a 3-dimensional translation command, a 3-dimensional rotation command and a 1-dimensional gripper command. It is not a set of seven joint angles, and the numbers cannot be read directly as metres. At a 20 Hz control rate each step corresponds to roughly 0.05 s of simulated time; internal physics sub-steps are not counted as policy steps. A budget of 600 or 700 steps therefore corresponds to about 30 or 35 seconds of simulated time, which is unrelated to wall-clock runtime. The budget is the maximum number of control steps allowed — not a monetary or GPU-capacity limit. An episode stops as soon as the goal is satisfied; if the budget is exhausted the episode is recorded as a failure. The initial stabilisation wait is not charged against the policy budget.

Success is measured by the task-specific scoring rule, which inspects spatial relations, containment relations and joint or switch states of the objects. The policy sees the natural-language goal while the scorer reads the machine-checkable goal predicate. Success rate is defined as successful episodes divided by all episodes; with 50 trials per task and 500 per suite, and with equal denominators across the four suites, the mean of the suite scores equals the overall 2000-episode success rate. All failures are retained and engineering errors are reported separately; incomplete runs are never presented as full results.

Official summary scores are not a per-episode ground-truth archive. Agreement with the official number does not imply sample-by-sample identity, and because the two models reproduced in this report

use different native budgets, seeds, simulator versions and replanning rates, the difference between their scores cannot be interpreted as a fair architectural advantage.

1.3 Model and Inference Architecture

The checkpoint used here is `OpenWAM-Alpha-Sim-LIBERO/checkpoint_step_10690.safetensors`, pinned to Hugging Face revision `110b07e4d5581c81520a77419cbb0e9371df231b`. All nine files (24,835,230,170 bytes in total) were verified; the main weight file is 24,813,767,464 bytes with SHA-256 `34c9a6c9af67dda9aaffa2edc0efa29b28eeee1c26dea419f98065c80508c96a`.

The loading log confirms a Wan video DiT backbone, the WanVideoVAE38 decoder, a Wan text encoder and a dedicated action expert, composed through the authors' *Dual-System Joint Self-Attention* with `bfloat16` mixed precision. A Wan2.2-TI2V-5B video base [4] and the action branch are combined by joint self-attention, so that a video-generation backbone participates in world/action modelling — a design that differs from the vision-language backbone plus action expert used by $\pi_{0.5}$ (Experiment B). The dual-system joint modelling and the unified action representation are the authors' contributions, not contributions of this report.

Internally the model works in an 80-dimensional unified action representation. A normaliser maps this representation to and from a native 10-dimensional end-effector action, and the client converts the rotation part into LIBERO's 7-dimensional control, so the first seven entries cannot simply be sliced off.

Figure 1 lists the runtime data flow, which is a deployment diagram rather than a layer-by-layer network topology.

task language → author tokenizer / text encoder;
external + wrist RGB → two-view canvas / video encoder;
EEF 10-D state → min-max normalisation / 80-D mapping;
(text, video, state) → joint denoising of video backbone and action expert;
→ native EEF 10-D action → rotation conversion → 7-D LIBERO control;
→ execute and re-observe (feedback loop).

Figure 1 Runtime data flow of the OpenWAM-α closed loop.

The server logs report a 384×320 canvas with multi-view enabled; two cameras are actually present and the third slot is empty. Denoising uses a 10-step schedule in synchronous mode with a deployment horizon of 10. The authors additionally enable a DiT velocity cache with a threshold of 0.990, so the ten denoising steps must not be described as ten independent full video-DiT forward passes.

What is evaluated here is action success rate, not future-video quality. The full run stored no predicted future RGB and no future ground truth, so no world-model video metric, counterfactual accuracy or causal effect is reported. The fact that the architecture is a world–action model does not by itself demonstrate such capabilities in this experiment.

1.4 Closed-Loop Data Pipeline

A single episode proceeds as follows.

1. Task definitions, language, scene assets and initial states are read from the pinned LIBERO repository. Judging the released weights does not require the full training demonstration set, and this run did not download it.

2. One simulator instance is created per task and trials 0–49 are executed consecutively; after each reset the corresponding initial state is applied. The environment seed is 42.

3. Thirty stabilisation steps are executed and are not charged against the policy budget (600 for Spatial/Object/Goal, 700 for LIBERO-10).

4. External and wrist RGB images and the robot state are captured. The author-side end-effector state is a 3-D position, a continuous 6-D rotation and a 1-D gripper value. Object ground-truth fields exist in the simulator observation but were not fed to the policy.

5. The client packages images, task and state and sends the PNG payload over a WebSocket to a separate model process. The teaching hook records the genuine PNG bytes; no camera layout or $\pi_{0.5}$-style state adaptation is substituted.

6. The server applies min-max normalisation from the checkpoint's `normalization_stats.npy`, runs the joint denoising, and hands the output to the adapter.

7. The client converts the 6-D rotation of the native 10-D action into LIBERO rotation control to form a 7-D action and executes it at the author's caching cadence; typically a 10-step chunk is consumed before replanning. The 74-control-step teaching case corresponds to 8 planning calls — control steps are not neural inference calls.

8. Each action changes the scene and the next observation re-enters the loop. `env.done` is the success signal; episodes that time out are still counted in the denominator.

9. Per-task `results.json` files and the client `RESULT` log store every trial, its success flag, policy steps and terminal reward; the run is then summarised into `summary.json` and `summary.csv`.

Debug logging was disabled for the full run. Ninety log, configuration and result files were retained, but per-query PNGs, action chunks and states were not stored for the main table. A single teaching case additionally stores 74 genuine observation/action frames; it is a learning exhibit and does not replace the 2000-episode result.

1.5 Frozen Environment and Provenance

Table 1 Frozen configuration of the full OpenWAM-α run.

Item	Value
Author source	`48bd67b89d489b14d03b8d92bc66e65d306df32e`
LIBERO source	`8f1084e3132a39270c3a13ebe37270a43ece2a01`
Python (model/client)	3.11.16
PyTorch (model)	2.5.1+cu121
Transformers	5.5.0
NumPy	1.26.4
MuJoCo / robosuite	3.3.2 / 1.4.0
dtype / attention	`bfloat16` / Torch SDPA
`compile`	false
TF32	off, precision *highest*, `NVIDIA_TF32_OVERRIDE=0`
Hardware	single A100, EGL, model and simulator in separate processes
Model server replicas	1, port 8920, concurrency unchanged mid-run
Environment seed / trial	42 / 0–49 per task
Checkpoint saving	none (inference only, no new weights)

The authors recommend Python 3.10 and Torch 2.7; this run reuses an existing Python 3.11 / Torch 2.5.1 environment with SDPA and `compile` disabled. These deviations are disclosed and no claim of bitwise equality, equal speed or a fully reproduced author environment is made. System drivers, CUDA, the base Conda environment and the global shell were left untouched.

The formal run ID is `openwam-w2-libero-native-full-s42`. The pre-registered source Git revision was `0c6e95571585a29914f62e7036768be57379482d` and the queue file SHA-256 was `1078f11d00808fe962c6215e03f577499e219136dcb0d9cf6b1140bb495a25f`. The executed launch command was:

```
server-python OpenWAM/benchmarks/libero/run_all_suites.py \
  --ckpt-dir <models>/OpenWAM-Alpha-Sim-LIBERO \
  --ckpt-name checkpoint_step_10690.safetensors \
  --server-python <server-python> --libero-python <client-python> \
  --libero-path <LIBERO-8f1084e3...> \
  --policy-config <OpenWAM>/benchmarks/libero/policy_config.yml \
  --gpus 0 --render-gpus 0 --replicas-per-gpu 1 \
  --compile-enabled false --client-max-attempts 1 \
  --output-dir <runs>/openwam-w2-libero-native-full-s42/output
```

Re-evaluating the policy requires a new unique run ID, a fresh space/ETA registration and a new configuration; this output must not be overwritten.

1.6 Results

The full 2000-episode run completed with 1985 successes and 15 failures, giving an overall success rate of 99.25 %. Table 2 breaks the result down by suite and compares it with the authors' reported numbers.

Table 2 Per-suite success rate: this reproduction versus the official OpenWAM-α scores.

Suite	Success / total	This run	Official	Difference (pp)
libero_spatial	497 / 500	99.40 %	99.60 %	−0.20
libero_object	498 / 500	99.60 %	99.60 %	+0.00
libero_goal	497 / 500	99.40 %	99.80 %	−0.40
libero_10	493 / 500	98.60 %	98.20 %	+0.40
Mean / total	**1985 / 2000**	**99.25 %**	**99.30 %**	**−0.05**

The official table publishes only summary scores and provides no per-episode predictions, so sample-by-sample agreement with the authors cannot be claimed. The value 99.25 % is reported to two decimals and is not described as exactly matching 99.30 % merely because of rounding. A single fixed seed cannot demonstrate multi-seed stability, statistical superiority or a new method. Figure 2 shows the aggregate comparison and Figure 3 the per-task breakdown.

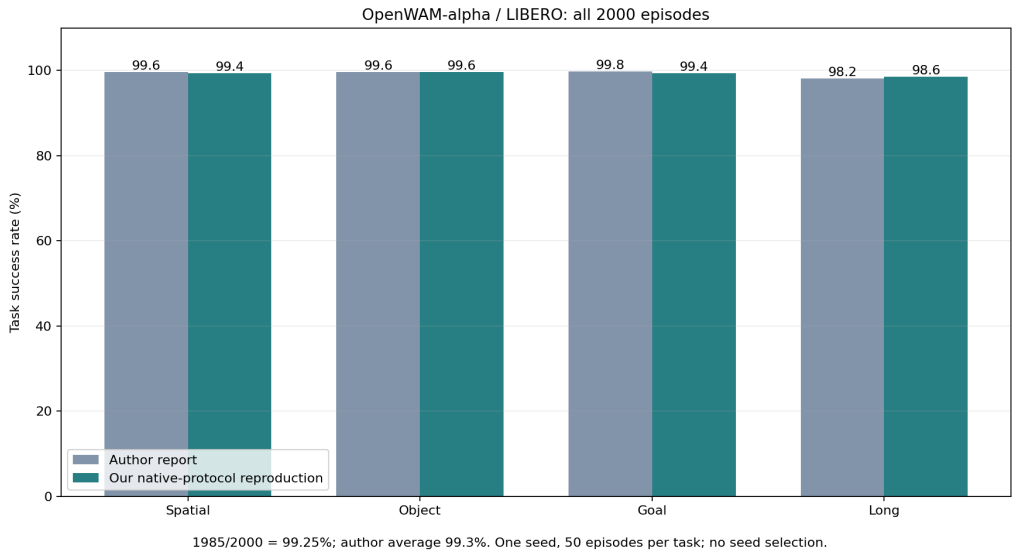


Figure 2 Official OpenWAM-α scores versus this reproduction.

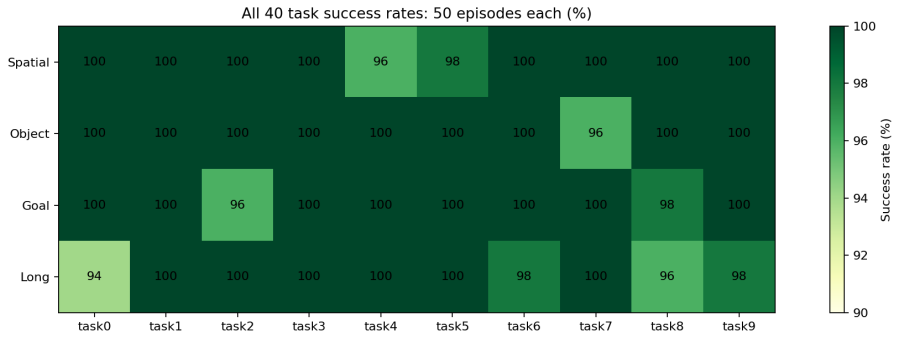


Figure 3 Per-task success rate over all 40 LIBERO tasks.

Table 3 lists every task together with the number of successful episodes and the failing trial IDs.

Table 3 Per-task results for the full OpenWAM-α run.

Suite	ID	Task language	Succ./50	Failing trials
libero_spatial	0	pick up the black bowl between the plate and the ramekin and place it on the plate	50	—
libero_spatial	1	pick up the black bowl next to the ramekin and place it on the plate	50	—
libero_spatial	2	pick up the black bowl from table center and place it on the plate	50	—
libero_spatial	3	pick up the black bowl on the cookie box and place it on the plate	50	—
libero_spatial	4	pick up the black bowl in the top drawer of the wooden cabinet and place it on the plate	48	4, 27
libero_spatial	5	pick up the black bowl on the ramekin and place it on the plate	49	2
libero_spatial	6	pick up the black bowl next to the cookie box and place it on the plate	50	—
libero_spatial	7	pick up the black bowl on the stove and place it on the plate	50	—
libero_spatial	8	pick up the black bowl next to the plate and place it on the plate	50	—
libero_spatial	9	pick up the black bowl on the wooden cabinet and place it on the plate	50	—
libero_object	0	pick up the alphabet soup and place it in the basket	50	—
libero_object	1	pick up the cream cheese and place it in the basket	50	—
libero_object	2	pick up the salad dressing and place it in the basket	50	—
libero_object	3	pick up the bbq sauce and place it in the basket	50	—
libero_object	4	pick up the ketchup and place it in the basket	50	—
libero_object	5	pick up the tomato sauce and place it in the basket	50	—
libero_object	6	pick up the butter and place it in the basket	50	—
libero_object	7	pick up the milk and place it in the basket	48	10, 13
libero_object	8	pick up the chocolate pudding and place it in the basket	50	—
libero_object	9	pick up the orange juice and place it in the basket	50	—
libero_goal	0	open the middle drawer of the cabinet	50	—
libero_goal	1	put the bowl on the stove	50	—
libero_goal	2	put the wine bottle on top of the cabinet	48	38, 47
libero_goal	3	open the top drawer and put the bowl inside	50	—
libero_goal	4	put the bowl on top of the cabinet	50	—
libero_goal	5	push the plate to the front of the stove	50	—
libero_goal	6	put the cream cheese in the bowl	50	—
libero_goal	7	turn on the stove	50	—
libero_goal	8	put the bowl on the plate	49	25
libero_goal	9	put the wine bottle on the rack	50	—
libero_10	0	put both the alphabet soup and the tomato sauce in the basket	47	7, 14, 38
libero_10	1	put both the cream cheese box and the butter in the basket	50	—
libero_10	2	turn on the stove and put the moka pot on it	50	—
libero_10	3	put the black bowl in the bottom drawer of the cabinet and close it	50	—
libero_10	4	put the white mug on the left plate and put the yellow and white mug on the right plate	50	—
libero_10	5	pick up the book and place it in the back compartment of the caddy	50	—
libero_10	6	put the white mug on the plate and put the chocolate pudding to the right of the plate	49	23
libero_10	7	put both the alphabet soup and the cream cheese box in the basket	50	—
libero_10	8	put both moka pots on the stove	48	29, 38
libero_10	9	put the yellow and white mug in the microwave and close it	49	25

Independent verification read all 40 `results.json` files and matched each task against the 50 client `RESULT` entries on trial ID, success flag and policy steps, additionally checking seed, stabilisation steps, budget, end-effector mode, coverage, reward, summary and manifest. All 2000 unique trials, 1985 successes and 15 failures agree. The 90 formal files (10,271,072 bytes) were returned with per-file size/SHA checks and a server acknowledgement fingerprint of `ca759e55b79bb6c7556399a685662773481eb07f91d908fd62c6d11d4cf91f21`. Every complete episode retains only its task, trial, outcome, action count and terminal reward — not the full action sequence, frame-by-frame media or environment state.

1.7 Compute, Timing and Control Budget

The formal run started at 2026-10-05T18:40:58+08:00 and finished at 2026-10-06T01:51:27+08:00 with exit code 0. The total wall time was 25,829.198 seconds (7 h 10 m 29 s), including model loading, compilation and caching, simulation, serialisation, task switching and log writing, but excluding model download, earlier failed attempts and document preparation.

Full-device sampling at 2-second intervals yielded 12,617 points, with a peak memory of 25,676 MiB ($\approx$ 25.07 GiB) and an average GPU utilisation of 29.40 %. A momentary high utilisation does not mean the GPU was saturated for the whole run, and the sampled peak may miss short spikes between samples.



Figure 4 Measured GPU memory and utilisation over the full run.

A single model replica used about 25 GiB, which suggests that two replicas would fit on the device, but whether that would reduce wall time still requires a controlled measurement under a fixed protocol; a linear speed-up is not promised. Any future increase in concurrency must record the model's random streams and task assignment and must not silently change the replica count of this frozen run.

Table 4 Executed control steps per suite.

Suite	Total control steps	Median per episode
libero_spatial	51,855	100.5
libero_object	66,294	129.0
libero_goal	56,520	92.0
libero_10	127,141	238.0
Total	**301,810**	—

The 301,810 executed control steps are not equal to the number of model queries; the full run has no per-query audit, so no exact call count or pure-GPU throughput is invented.

1.8 Failure Inventory

All 15 failures are listed in Table 5. Each exhausted its budget (600 steps for Spatial/Object/Goal, 700 for LIBERO-10) and ended with a terminal reward of 0.0.

Table 5 All 15 failed episodes of the full run.

Suite	Task	Trial	Control steps	Terminal reward
libero_spatial	4	4	600	0.0
libero_spatial	4	27	600	0.0
libero_spatial	5	2	600	0.0
libero_object	7	10	600	0.0
libero_object	7	13	600	0.0
libero_goal	2	38	600	0.0
libero_goal	2	47	600	0.0
libero_goal	8	25	600	0.0
libero_10	0	7	700	0.0
libero_10	0	14	700	0.0
libero_10	0	38	700	0.0
libero_10	6	23	700	0.0
libero_10	8	29	700	0.0
libero_10	8	38	700	0.0
libero_10	9	25	700	0.0

The failures span drawer retrieval, dairy placement, placing a bottle on a cabinet, putting a pot on the stove and combined operations, but a task name is not a diagnosed failure mechanism. Without the full video and action record of each failing episode, the failures cannot be attributed to occlusion, memory, grasping, planning or model hallucination. Any future diagnosis must register a new run and new random conditions; new footage from a different random stream cannot be presented as these 15 original failure trajectories.

1.9 Reproducibility Notes

The first smoke attempt failed because the author implementation rejected the `torch_sdpa` attention enum, producing 0 episodes; after switching to the legal `torch` value, a second attempt still produced 0 episodes because `pandas` was missing during normaliser import. Both failures and their logs were kept.

The root cause is that three packages eagerly re-export a training-only LeRobot reader alongside the inference normaliser. Because the private disk lacked room to install the full `pandas`/`pyarrow` training stack, only the three initialisers were changed to lazy loading in a separate copy of the author source; the remaining 546 files (including the model and the normalisation maths) are SHA-identical, and a real CPU normaliser import and min–max round trip passed. The tool is provided as `prepare_inference_imports.py`. On 2026-10-06 the same 546 files and three initialisers were re-verified.

The teaching retry then succeeded (1/1, 74 steps) and the full run completed once with exit code 0 and `client_max_attempts=1`; no best-of-retry results were spliced into the main table. TCP readiness probes during server start-up produced `InvalidMessage` handshake log lines, which are not failed policy tasks — log lines containing the word "error" are not automatically treated as NaN.

1.10 Teaching Case: Images, Actions and Video

The teaching case fixes Spatial task 0, initial state 0 and succeeded once in 74 control steps with 8 planning calls. All native 10-D end-effector, 7-D LIBERO and 10-D state values are finite. It is not an extra 2001st episode of the main table. The input images are the data the model genuinely received, not paper screenshots or generated images.

The state is a 3-D position, a 6-D rotation and a 1-D gripper value ($3 + 6 + 1 = 10$); the internal 80-D representation is a unified model format, not 80 robot degrees of freedom. The following record is

the genuine first control step of the teaching case. Note that the native action and the final control differ in rotation and gripper due to adaptation, and that the final 7-D action is not the first seven entries of the native 10-D action.


```
{
  "state": [
    -0.21130245923995972, -0.010944196954369545, 1.1741831302642822,
    0.9984198808670044, -0.00016182575200218707, -0.056193094700574875,
    -0.0001631764171179384, -1.0, -1.944792347785551e-05, 0.975337028503418
  ],
  "native_eef10_action": [
    0.3204345405101776, 0.3149413764476776, 0.1583862155675888,
    0.99609375, 0.0078125, -0.0302734375, -0.0146484375,
    0.99609375, -0.0087890625, 1.0078125
  ],
  "libero_action7": [
    0.3204345405101776, 0.3149413764476776, 0.1583862155675888,
    -0.009145314805209637, 0.030346114188432693, 0.007979447022080421, -1.0
  ]
}
```


Figure 5 shows the two genuine input views of the case, and Figure 6 plots the executed actions together with the end-effector position.

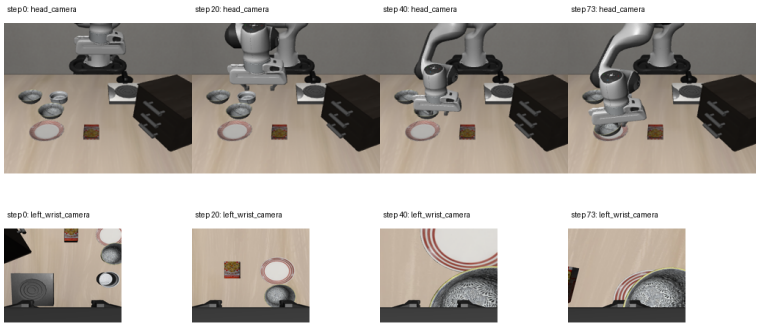


Figure 5 Two genuine input views of the teaching case.

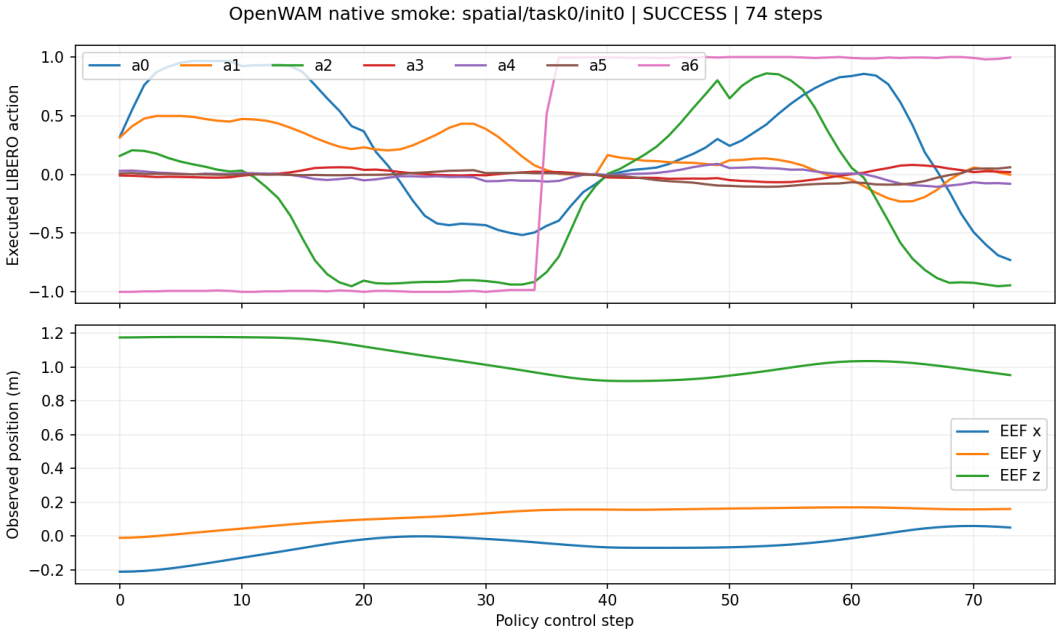
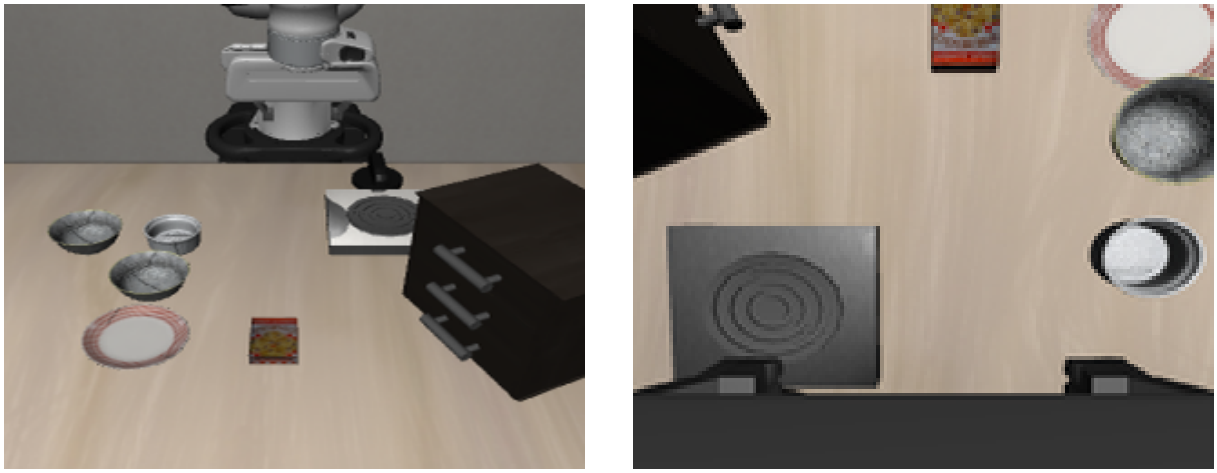


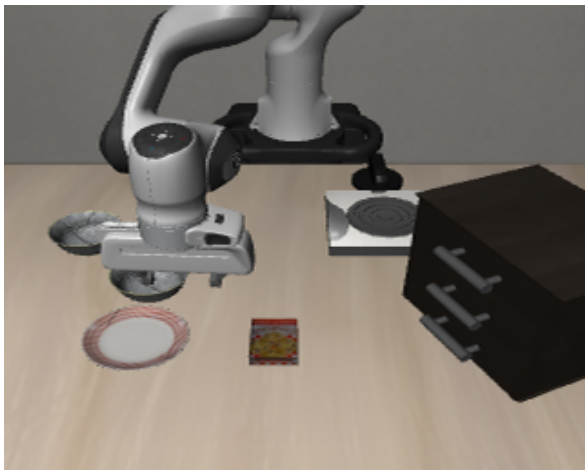
Figure 6 Executed actions and end-effector position of the teaching case.

Figures 7, 8 and 9 show the two camera views at three points of the episode: before the first action, at step 36 and at step 73. The saved frames precede their actions, and success is decided by the post-action environment signal rather than by the last image.

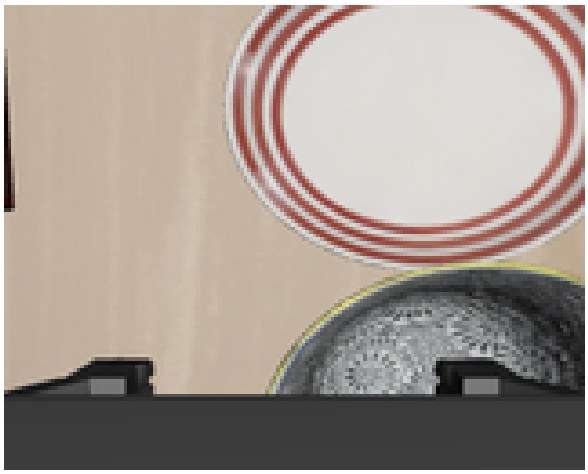


(a) external view **(b)** wrist view

Figure 7 Observation before the first action (step 0).

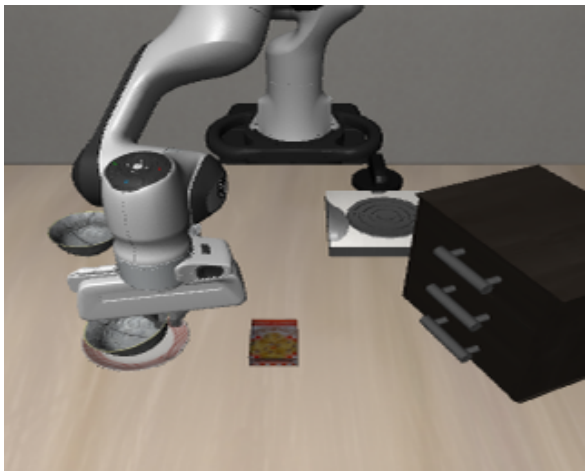


(a) external view

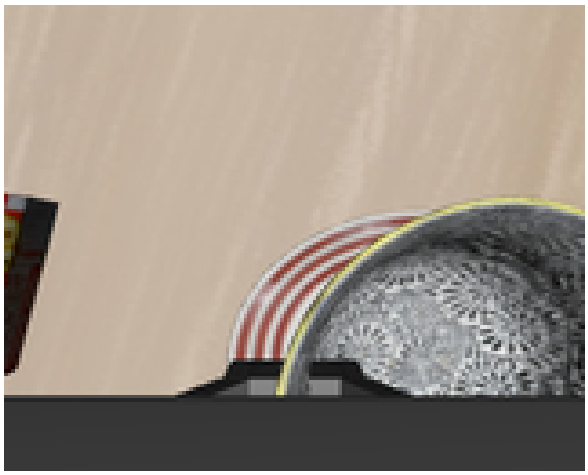


(b) wrist view

Figure 8 Observation before the action at step 36.



(a) external view



(b) wrist view

Figure 9 Observation before the last action (step 73).

The corresponding 74-step two-camera video is stored alongside the figures as `case-rollout.mp4` and can be opened in a local player. It is the genuine smoke case from before the full run, not an author demo or a generated video, and not an original recording of the W2 episodes. The `head_camera` field is the simulator's external camera, not a human-worn camera.

1.11 Discussion: Reading OpenWAM Alongside $\pi_{0.5}$

Table 6 places the two reproductions side by side.

Table 6 OpenWAM-α versus $\pi_{0.5}$, each under its own native protocol. The two columns answer separate reproduction questions.

Item	$\pi_{0.5}$ (Exp. B)	OpenWAM-α (Exp. A)
Mean success rate	96.75 %	99.25 %
Official mean	96.85 %	99.30 %
Offset from official	−0.10 pp	−0.05 pp
Environment seed	7	42
Stabilisation wait	10	30
Spatial/Object/Goal budget	220/280/300	600/600/600
LIBERO-10 budget	520	700
Execution per plan	predict 10, execute first 5	native cache cadence, horizon 10
MuJoCo	3.2.3	3.3.2
Per-query audit	saved and independently verified	debug off, no equivalent record

The two evaluations share the same four LIBERO suites, but the models, initial random conditions, time budgets, control cadence and simulator versions were not unified. The two columns therefore answer each author's own reproduction question, and the difference $99.25\,\% - 96.75\,\% = 2.50$ percentage points must not be presented as a fair architectural gain. A model-to-model comparison would require a separate pre-registered protocol that fixes the environment, seed, initial states, budget, action cadence and cost. The high score obtained here is a baseline reproduction, not a new method.

1.12 Limitations

The claims of this experiment are bounded as follows. Evaluation is closed-loop action success in simulation only; no real-robot deployment is attempted. World-model video quality is not measured. Only one seed and one initial-state set per task are used, so multi-seed stability is not established. The full run keeps no per-query media, which limits post-hoc failure analysis. The environment differs from the authors' recommendation in Python and Torch versions. The official numbers are summary scores with no per-episode archive, so exact agreement with the authors cannot be verified.

1.13 Summary

On a single A100 with the authors' released weights and native protocol, OpenWAM-α reproduces $1985/2000 = 99.25\,\%$ on the four LIBERO suites, 0.05 percentage points below the official $99.30\,\%$. All 15 failures are retained, the environment and provenance are documented, and a genuine teaching case is provided with images, actions and a video. The result is a usable, verifiable baseline of the authors' policy together with engineering and reproducibility experience — not a new contribution, and not automatically transferable to human egocentric video.

2 Experiment B: $\pi_{0.5}$ on LIBERO

2.1 Overview and Scope

This experiment reproduces the official four-suite LIBERO success rate of Physical Intelligence's $\pi_{0.5}$ policy [1] on a single A100 GPU. The authors' released LIBERO fine-tuned weights are evaluated with the OpenPI reference implementation; no new $\pi_{0.5}$ model was trained, no RoboDojo-specific weights were substituted, and no video-classification score is used in place of a robot task success rate.

The main protocol, labelled P12, uses the authors' default JAX precision. A separate control run P10 uses the highest precision with TF32 disabled and reaches $97.00\,\%$; it is reported as a stability and precision control only and does not replace the pre-registered main protocol. The P labels are internal stage tags, not model versions: *rpc* denotes a separated model/simulation process, *native* the authors' default protocol and *full* a 2000-episode run.

The evaluation covers the four standard suites — Spatial, Object, Goal and LIBERO-10 — each with 10 tasks and 50 episodes per task, for $40 \times 50 = 2000$ closed-loop episodes in total. The complete

statistics, the per-task breakdown and representative teaching cases are described below.

2.2 Benchmark and Evaluation Protocol

LIBERO is a language-conditioned manipulation benchmark for simulated robot arms [2]. Each task defines a tabletop scene, a natural-language goal and a BDDL goal predicate; the policy observes the current RGB frames and the robot state, predicts an action, and MuJoCo advances the scene to produce the next observation until the goal is met or the budget is exhausted. This is a closed loop in which the robot's own actions change the next frame — it is not one-shot captioning of a recorded video.

Table 7 The four LIBERO suites and the official $\pi_{0.5}$ scores used as the comparison target. Official numbers come from the OpenPI release table [1], not from this run.

Suite	Focus	Tasks	Per task	Max steps	Official
Spatial	spatial relations in language	10	50	220	98.8 %
Object	different target objects	10	50	280	98.2 %
Goal	different goal states	10	50	300	98.0 %
LIBERO-10	longer-horizon combined tasks	10	50	520	92.4 %
Total / mean	four suites, equal weight	40	2000 total	—	96.85 %

The scalar values used in this report have precise meanings. The final LIBERO action is a 3-D translation command, a 3-D rotation command and a 1-D gripper command; it is not a set of joint angles and the numbers cannot be read as metres. At 20 Hz each control step is about 0.05 s of simulated time, so a 220-step budget is roughly 11 seconds of simulated time and is unrelated to wall-clock runtime. Success rate is successful episodes over all episodes; all failures are retained and no run is presented as complete unless every episode finished.

The state vector is an end-effector position (3), an axis-angle orientation (3) and two gripper joint values (2), for a total of 8 dimensions. The axis-angle vector encodes the rotation axis in its direction and the angle in its length; it is not a set of Euler angles. A positive or negative gripper value is fixed by the joint-axis convention. As explained in Section 2.3, this particular LIBERO configuration does not feed the state into the network, so the interface field must not be mistaken for a model input.

2.3 Model and Configuration

$\pi_{0.5}$ is a vision-language-action model: its interface receives a language task, two camera RGB streams and the robot state, and emits a continuous action chunk. It is not treated here as a world model, because this evaluation produces actions and generates no future RGB. The official configuration used is `Pi0Config(pi05=True, action_horizon=10, discrete_state_input=False)`; the concrete module implementation lives in the pinned files `src/openpi/models/pi0.py` and `pi0_config.py`, and no network is invented beyond them.

Source review clarifies an important nuance. `embed_prefix` consumes the image and task tokens, while `embed_suffix` creates a state token only when `not self.pi05`. Because this configuration additionally sets `discrete_state_input=False`, the discrete state text is disabled as well, so this LIBERO weight does not use the state as a neural condition. The 8-D state is still transformed by the interface, stored and used for batch shape inference. Receiving a state at the interface is therefore not the same as using it, and this conclusion must not be generalised to other $\pi_{0.5}$ weights whose default configuration includes a discrete state.

The fixed implementation uses a SigLIP `So400m/14` vision encoder, a `gemma_2b` language backbone and a `gemma_300m` action expert; these names are source configuration labels and are not a full parameter inventory. Vision tokens and the task text form the prefix; the action expert maps noise to continuous actions and injects the flow-matching time through adaRMSNorm. The internal action dimension is 32 with a horizon of 10, and the final adapter takes the first 7 dimensions. The maximum token length is 200 and the missing third camera is masked out; only two real cameras are present. Each query starts from

Gaussian noise and is denoised by the authors' default 10 flow-matching updates; the first 5 steps of the chunk are executed before replanning.

Intuitively, the model predicts *how to move next*, not *what the image becomes after the move*. Training such a VLA therefore needs paired camera, instruction and continuous robot action data; action categories from human first-person video are not a substitute for 7-D arm control targets. This work reproduces inference and evaluation only and does not retrain the backbone or the action expert.

2.4 Closed-Loop Data Pipeline

1. The tasks are read in a fixed suite order from the pinned LIBERO BDDL and language definitions. Task descriptions are never reworded to suit the model.
2. After `env.reset()` the author-provided initial state with index 0–49 is applied; the environment seed is fixed at 7.
3. Ten steps of `[0,0,0,0,0,0,-1]` are executed so that objects settle. These ten steps are not charged against the policy budget.
4. 256×256 RGB frames are read from `agentview_image` and `robot0_eye_in_hand_image` and flipped along both axes (equivalent to a 180° rotation), matching the authors' training preprocessing.
5. The frames are aspect-preservingly padded to 224×224 as `uint8`. Both real cameras are kept; the missing third camera is zero-filled and masked in the official adapter.
6. The end-effector position (3), the quaternion converted to axis-angle (3) and the gripper joints (2) form the 8-D state. It is genuine robot proprioception; this configuration does not encode it as a network condition but records it for the interface and the audit.
7. Quantile normalisation uses the statistics shipped with the checkpoint, the tokenizer encodes the task text, and the state is padded to the model's internal dimension.
8. A 10×7 action chunk is predicted and checked for shape and finite values. The de-normalised output is executed directly, with no extra gripper inversion and no action format borrowed from another robot.
9. Observations and predictions are refreshed every 5 steps. All predicted chunks, executed actions, states, rewards, `done` flags and the independent goal check are retained.
10. The official `done` signal is the primary success criterion and `env.check_success()` is a step-wise diagnostic. Reaching the maximum number of steps without success is recorded as a failure and is never removed from the denominator.

A single query produces 10 steps but only the first 5 are executed before replanning. PNG files are therefore indexed by query, while the video and `steps.json` are indexed by the actual control step; the 5 unexecuted steps remain predictions and must not be read as executed actions. The action components a_0–a_5 are controller commands and a_6 is the gripper command, so they cannot be interpreted directly as end-effector metres.

Figure 10 summarises the runtime flow.

two current cameras (256 RGB) $\rightarrow$ 180° rotation and 224 padding;
EEF position / orientation + gripper 8-D state $\rightarrow$ quantile normalisation and padding;
task language $\rightarrow$ tokenizer;
(images, text) $\rightarrow$ vision-language backbone $\rightarrow$ action expert with 10 flow-matching steps;
$\rightarrow$ 10×7 action chunk $\rightarrow$ execute first 5 $\rightarrow$ new MuJoCo observation and `done`.

Figure 10 Runtime data flow of the $\pi_{0.5}$ closed loop. The 8-D state is retained by the interface for the audit but is not a network condition in this configuration.

2.5 Frozen Source and Environment

Table 8 Frozen source and environment of the main (P12) protocol.

Item	Value
OpenPI revision	`215abfb217dbac7d5f1273282331b9b1866c0479`
LIBERO submodule	`f78abd68ee283de9f9be3c8f7e2a9ad60246e95c`
Runtime	JAX implementation, parameters restored as `bfloat16`
Python environment	Python 3.11, dependencies in isolated per-run trees
MuJoCo / robosuite	3.2.3 / 1.4.1
Rendering	A100 via EGL (dispatcher unpacked into the project, loaded per subprocess)
Environment seed	7
Policy seed	0, one independent policy per suite
Initial states	the authors' fixed states 0–49 per task
Action execution	predict 10, execute first 5, then replan

The server reuses an existing Python 3.11 environment with dependencies installed under isolated `runtime/` trees, without modifying the base environment. MuJoCo 3.2.3 and robosuite 1.4.1 match the official lock file consulted. Python, Torch and NumPy versions differ from the authors' older client environment, so bitwise identity is not claimed.

EGL lets the A100 run MuJoCo without IsaacSim's RT-core requirement. The server originally lacked the EGL dispatcher, so Ubuntu's official `libegl1`/`libopengl0` packages were unpacked into the project's `system-libs` directory and loaded only through a subprocess `LD_LIBRARY_PATH`; no system package was installed and no driver was changed. Version 2.3.7 was used only for a rendering probe; the policy cases and the full evaluation all used 3.2.3.

The main-protocol run IDs are `pi05-p12-rpc-native-full-{spatial,object,goal,10}-s7`. The model and simulation processes are launched through a project launcher that sets up the isolated dependencies and the EGL environment; the launcher refuses to reuse an existing run directory, so rerunning requires a fresh unique run ID, a free port and a new space/ETA registration. The command form is:

```
python launch_rpc_run.py \
  --config <runtime>/execution/<git>-pi05-native/spatial-config.json
```

Each run records both subprocess commands and the explicit environment overrides in `command.json`, the frozen configuration, source fingerprints, seed and budget in `prereg.json`, and the exit code and total wall time in `status.json`.

2.6 Results

The main P12 protocol completed all 2000 episodes and was independently verified: $1935/2000 = 96.75\,\%$ versus the official $96.85\,\%$, a difference of -0.10 percentage points. All 64,761 inferences and 320,142 executed action steps were checked for finite values, with no invalid prediction.

Table 9 Per-suite results. P12 is the pre-registered main protocol at the authors' default precision; P10 is a separate highest-precision control and does not replace the main table.

Suite	Success / total	P12	Official	Difference (pp)	P10 control
spatial	495 / 500	99.00 %	98.80 %	+0.20	97.60 %
object	490 / 500	98.00 %	98.20 %	−0.20	98.60 %
goal	491 / 500	98.20 %	98.00 %	+0.20	98.00 %
LIBERO-10	459 / 500	91.80 %	92.40 %	−0.60	93.80 %
Mean	**1935 / 2000**	**96.75 %**	**96.85 %**	**−0.10**	**97.00 %**

A precision control on the same Spatial inputs with policy seed 0 confirmed that the authors' default and the historical highest/TF32-off settings are not numerically identical: the maximum action difference was 0.007402592285156384. Because the official entry point does not set a JAX matrix precision, P12 keeps the authors' default precision and executes revision `c667a73`.

Figure 11 compares the official, main-protocol and precision-control scores, and Figure 12 shows the per-task rates.

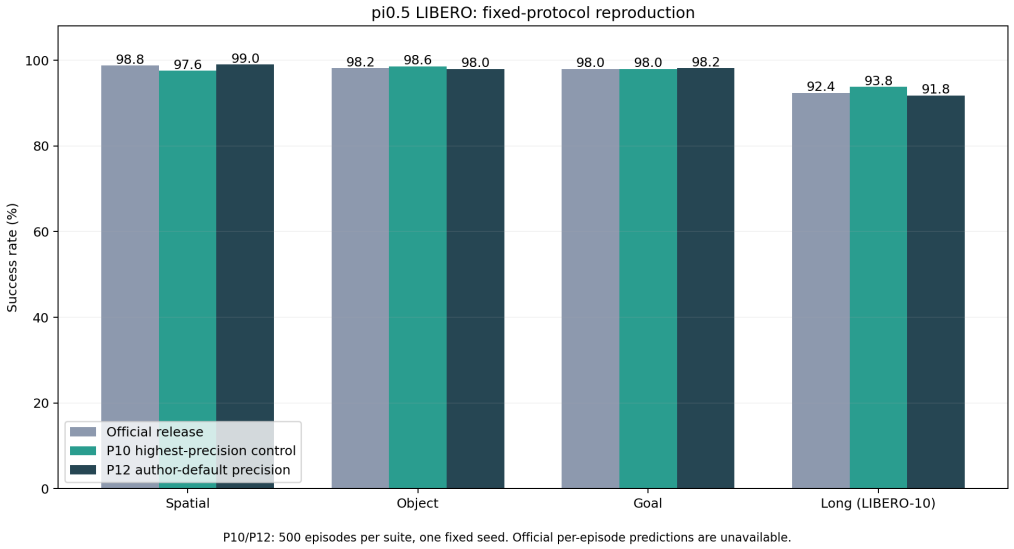


Figure 11 Official, main-protocol (P12) and precision-control (P10) scores.

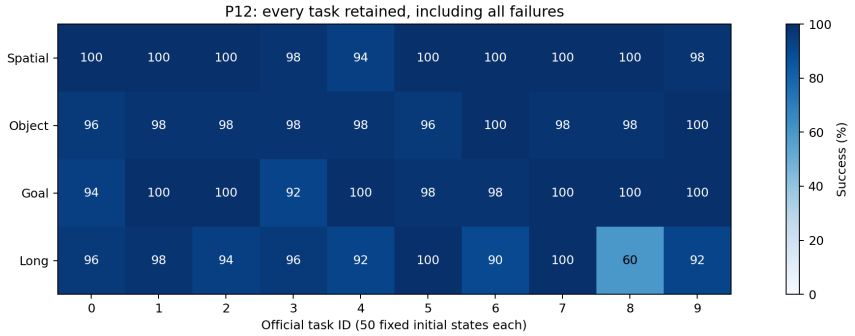


Figure 12 Per-task success rate over all 40 LIBERO tasks.

Table 10 Per-task results (50 episodes per row). All initial states are retained; the failing IDs are kept for auditing and must not be used to select inputs for a future evaluation.

Suite	ID	Task language	P12 /50	P10 /50	P12 failing IDs
spatial	0	pick up the black bowl between the plate and the ramekin and place it on the plate	50	50	—
spatial	1	pick up the black bowl next to the ramekin and place it on the plate	50	50	—
spatial	2	pick up the black bowl from table center and place it on the plate	50	50	—

continued on the next page

Table 10, continued

Suite	ID	Task language	P12 /50	P10 /50	P12 failing IDs
spatial	3	pick up the black bowl on the cookie box and place it on the plate	49	49	7
spatial	4	pick up the black bowl in the top drawer of the wooden cabinet and place it on the plate	47	47	23, 25, 46
spatial	5	pick up the black bowl on the ramekin and place it on the plate	50	47	—
spatial	6	pick up the black bowl next to the cookie box and place it on the plate	50	50	—
spatial	7	pick up the black bowl on the stove and place it on the plate	50	48	—
spatial	8	pick up the black bowl next to the plate and place it on the plate	50	50	—
spatial	9	pick up the black bowl on the wooden cabinet and place it on the plate	49	47	47
object	0	pick up the alphabet soup and place it in the basket	48	49	2, 44
object	1	pick up the cream cheese and place it in the basket	49	50	5
object	2	pick up the salad dressing and place it in the basket	49	48	38
object	3	pick up the bbq sauce and place it in the basket	49	49	33
object	4	pick up the ketchup and place it in the basket	49	48	7
object	5	pick up the tomato sauce and place it in the basket	48	50	7, 34
object	6	pick up the butter and place it in the basket	50	50	—
object	7	pick up the milk and place it in the basket	49	50	1
object	8	pick up the chocolate pudding and place it in the basket	49	50	16
object	9	pick up the orange juice and place it in the basket	50	49	—
goal	0	open the middle drawer of the cabinet	47	49	1, 19, 31
goal	1	put the bowl on the stove	50	50	—
goal	2	put the wine bottle on top of the cabinet	50	50	—
goal	3	open the top drawer and put the bowl inside	46	44	4, 20, 38, 45
goal	4	put the bowl on top of the cabinet	50	50	—
goal	5	push the plate to the front of the stove	49	50	44
goal	6	put the cream cheese in the bowl	49	49	49
goal	7	turn on the stove	50	50	—
goal	8	put the bowl on the plate	50	50	—
goal	9	put the wine bottle on the rack	50	48	—
libero_10	0	put both the alphabet soup and the tomato sauce in the basket	48	46	18, 49
libero_10	1	put both the cream cheese box and the butter in the basket	49	50	33
libero_10	2	turn on the stove and put the moka pot on it	47	50	6, 25, 33
libero_10	3	put the black bowl in the bottom drawer of the cabinet and close it	48	49	2, 13
libero_10	4	put the white mug on the left plate and put the yellow and white mug on the right plate	46	49	7, 13, 34, 35
libero_10	5	pick up the book and place it in the back compartment of the caddy	50	50	—
libero_10	6	put the white mug on the plate and put the chocolate pudding to the right of the plate	45	49	27, 30, 33, 38, 49
libero_10	7	put both the alphabet soup and the cream cheese box in the basket	50	49	—
libero_10	8	put both moka pots on the stove	30	31	0, 2, 5, 6, 7, 9, 13, 14, 20, 21, 23, 26, 28, 30, 36, 37, 38, 41, 42, 49
libero_10	9	put the yellow and white mug in the microwave and close it	46	46	3, 18, 44, 48

Acceptance checking compared, on the server, that no task or initial state was missing or duplicated, that every episode's success agreed with the `done/goal` signal, and that queries and actions matched the stored NPZ files. The complete inputs, states and videos stay on the server; in line with the delivery scope, only statistics, records and representative cases were returned locally. Seeds were not selected and runs were not retried until the score looked good, and agreement with the official number was not used as a tuning target. The official release publishes only summary scores and no per-episode predictions, so per-sample identity with the authors cannot be claimed.

2.7 Compute, Timing and Archive

P12 ran from the first suite's start to the last suite's exit in 9673.44 seconds (about 2 h 41 m 13 s). The four suites overlap, so their wall times must not be summed. This interval includes model loading, first compilation, rendering, file writing and concurrency waiting, but excludes earlier downloads, diagnostics, P10 and document preparation.

Table 11 Per-suite runtime and request statistics. RPC latency includes networking, serialisation, service queuing and actual inference, so it is not a pure GPU kernel time.

Suite	Wall time	Queries	Control steps	RPC median / P95 (s)
spatial	85.36 min	10,706	52,625	0.197 / 0.249
object	95.89 min	14,381	70,938	0.193 / 0.263
goal	85.43 min	11,660	57,342	0.211 / 0.265
LIBERO-10	147.45 min	28,014	139,237	0.094 / 0.254
Total	**9673.44 s**	**64,761**	**320,142**	—

At launch, five model/render process groups used roughly 46 GiB in total and then wound down as tasks finished. GPU sampling is whole-device, so a concurrent peak must not be reported as a single-model memory footprint. The lower latency in the second half of LIBERO-10 reflects reduced contention, not faster model computation.

Each run directory stores `summary.json` (500 episodes), `server/inference-audit.jsonl` (the finite-value check of every real model call), `prereg.json`, `command.json`, `status.json` and `gpu-samples.jsonl`. The returned local media contain, per episode:

Table 12 Per-episode files and their boundaries.

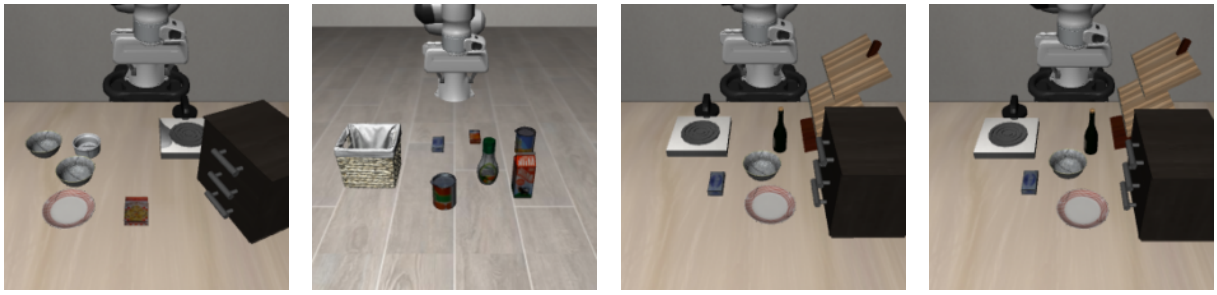
File	Content and boundary
`initial-state.npy`	the author initial state, used only for reset
`query-NNN-*.png`	the two 224 RGB inputs of one real query; not one PNG per control step
`query-NNN.npz`	the 8-D state and the predicted 10×7 action; the last 5 steps may be unexecuted
`queries.json`	every query index, control step, RPC wall latency and full prediction
`steps.json`	the pre-step state, executed action and post-step reward/ `done`/goal
`rollout.mp4`	the two RGB streams side by side, lossy; no extra terminal frame
`summary.json`	task language, outcome, step and query counts, input fingerprints

The action control rate is 20 Hz in simulation while videos are written at 10 fps for viewing, so a video lasts about twice the simulated action time. Wall time additionally includes model, rendering, file-writing and concurrency overhead; the three times must not be conflated. An archive is considered complete only when both `archive-receipt.json` (private logs and summaries) and `media/media-receipt.json` (per-file sizes and SHAs) exist and are verified.

2.8 Teaching Cases: Six Genuine Episodes

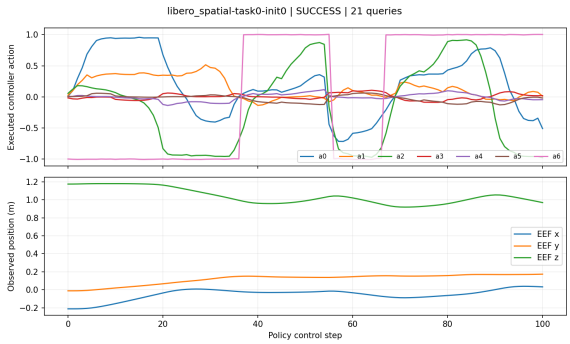
The six P9 engineering cases below share a random stream across suites and are not P12 main-table episodes; 5/6 success cannot stand in for the 2000-episode score. First- and last-query images are not post-termination ground truth: success is decided by the official `done/goal` signal after the action. The complete `queries.json` stores all predictions, including the unexecuted ones, while `steps.json` stores what was actually executed.

Figure 13 collects the first genuine model input (external and wrist view) of the spatial, object and goal cases, and Figure 14 shows the executed actions with the observed end-effector position for each case.

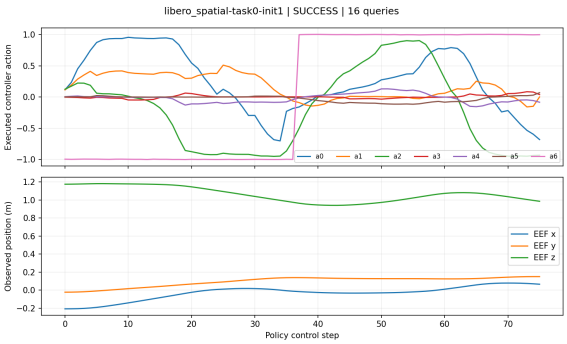


(a) spatial t0 i0 **(b)** object t0 i0 **(c)** goal t0 i0 **(d)** goal t0 i1

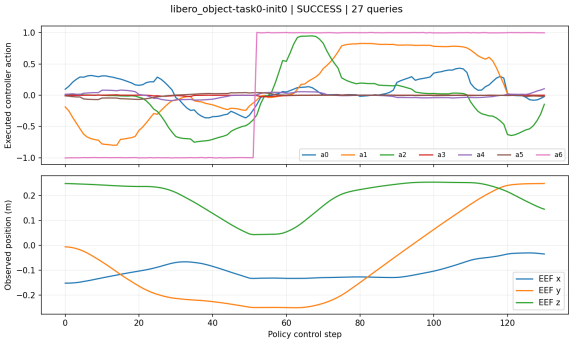
Figure 13 First genuine external-camera input of four of the six teaching cases. The wrist views of the same queries are provided in the accompanying media folder.



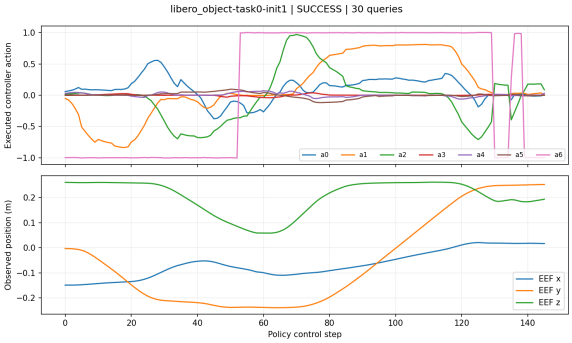
(a) spatial t0 i0 (success, 101 steps)



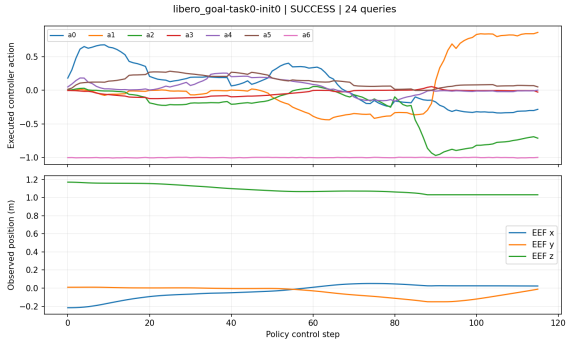
(b) spatial t0 i1 (success, 76 steps)



(c) object t0 i0 (success, 131 steps)



(d) object t0 i1 (success, 146 steps)



(e) goal t0 i0 (success, 116 steps)



(f) goal t0 i1 (timeout failure, 300 steps)

Figure 14 Executed actions and observed end-effector position for the six teaching cases.

The first genuine query of spatial task 0, initial state 0 is reproduced verbatim below. It is real data, not a hypothetical example.


```
{
  "query_id": 0,
  "step": 0,
  "state_8d": [
    -0.21124162683394807, -0.011090601889321502, 1.1742495176982402,
    3.140895396142254, -0.002847819833825949, -0.08773931027790592,
    0.03872336505859045, -0.0387222020954563
  ],
  "first_predicted_action_7d": [
    0.010095496548414129, 0.027211936503648726, 0.05642917198252673,
    -0.01643640754377028, 0.005886954390445509, 0.0053137199111878974,
    -0.9974632556626797
```

```
  ]
}
```

The position components are expressed in the simulation frame and are not heights above the table; the gripper sign follows the joint-axis convention. The state describes the present and the actions indicate the next step. Because this configuration does not use the state as a network condition, the report must not claim that the model consumed the 8-D state.

The six rollouts are stored alongside the figures as `<suite>-t0-i<i>-rollout.mp4`. The complete predictions and executed steps are provided as `queries.json` and `steps.json`, respectively. All inputs, predictions and videos were returned with per-file SHA checks (`media-receipt.json`) and the six action-curve PNGs (1,038,710 bytes in total) were verified as well. The action expert cannot be credited with general robot generalisation or a paper-level contribution on the basis of these six cases.

2.9 Numerical Anomalies and Non-Finite Failures

An early Object run stopped after the first 14 episodes of the P4 protocol when a non-finite action appeared at initial state 14, exiting with code 1; the partial run cannot be reported as a score and was retained as-is. A revision then added diagnostic saving, counted an invalid prediction as an episode failure and continued, without changing the gripper or the seed. The independent 500-episode retry was not spliced with the best results, and the supervisor process started a retry only once, for the actually failing suite.

A long-horizon diagnostic read the GPU input tensors after 2699 anomalous inferences and captured a device-side data anomaly: all three image tensors were NaN, the tokens were abnormally zeroed and the raw 320 action output was NaN, while the random noise was normal. Repeating with a fresh random key on the same input still failed. The fact of the anomalous GPU input is confirmed, but the underlying CUDA/EGL cause is not proven and must not be asserted. These non-finite failures are distinct from genuine task failures under normal numerical conditions.

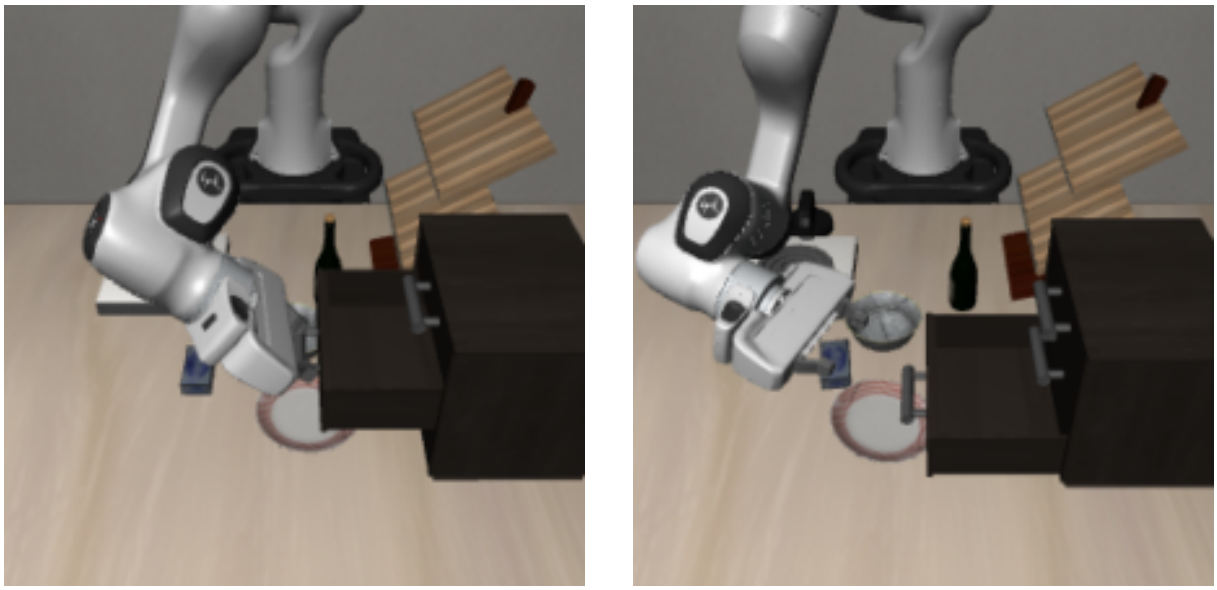
2.10 Normal Failures: Open Drawer, Still a Failure

The following two genuine episodes come from the P10 highest-precision control, not from the P12 main table. Both use the language "open the middle drawer of the cabinet", and the author BDDL goal `Open wooden_cabinet_1_middle_region` requires the middle-drawer joint coordinate to be below -0.14.

The original actions of 127 and 300 steps were replayed on the CPU without rendering, in the original within-task environment reset order. For both episodes the maximum difference over all 8-D end-effector states was 0, and the step-wise `done` and `goal` signals agreed. The joint values below therefore come from a verified physical replay.

Table 13 Physical replay of the two drawer episodes.

Case	Middle drawer	Bottom drawer	Official goal
task 0 / init 0	-0.14426769	0.0	success
task 0 / init 1	0.00147690	-0.15999917	failure (bottom drawer opened)



(a) successful episode, last query

(b) failed episode, last query

Figure 15 The last genuine query of the successful and the failing drawer episode. A visible open drawer does not prove task success.

A drawer is clearly open in both images, yet only one episode is successful. Object identity and goal state must both be satisfied, so a picture of an open drawer is not sufficient evidence of success. Every action in the failed episode is finite: it is a genuine task failure and must not be grouped with the earlier non-finite engineering anomalies.

The replay itself had failures worth recording. The first attempt used a wrong import path. The first retry created a new environment per episode and produced a maximum state error of 0.05557 on the failing episode, so it cannot be called an exact replay. The second retry preserved the official within-task environment creation and reset order and reduced the error to zero. Random sequence and internal controller state were not separated factor by factor, so no specific internal causal mechanism is asserted. All three records were returned with SHA and server acknowledgement and none was overwritten.

2.11 Limitations and Next Steps

The claim of this experiment is bounded accordingly. Evaluation is closed-loop action success in simulation only and involves no real-robot deployment. The official numbers are summary scores with no per-episode archive, so per-sample identity with the authors cannot be verified. The precision control compares single runs and no multi-seed study was performed, so the difference between P12 and P10 cannot be generalised into a stable precision advantage. The 8-D state is not a network condition in this configuration, which restricts conclusions about state-conditioned policies in general. Videos are written at 10 fps for viewing and the last partial action chunk may not be fully executed, which limits frame-by-frame interpretation.

For a more recent benchmark, *LIBERO-Para* provides 2026 language paraphrase tests with an official $\pi_{0.5}$ guide and is the natural next target once the current protocol is stable. *LIBERO-Plus* studies visual, environmental and language perturbations but needs extra assets and a different number of evaluation runs; the π_0 results in its tables must not be read as $\pi_{0.5}$. *RoboCasa365* is a broader kitchen-manipulation candidate but requires matching weights, cameras and control adaptation, and LIBERO weights must not be used to climb its leaderboard. The present result is a trustworthy baseline reproduction, not a new method, and does not automatically transfer to human first-person egocentric video.

2.12 Summary

On a single A100 with the authors' released LIBERO weights and the official OpenPI implementation, $\pi_{0.5}$ reproduces $1935/2000 = 96.75\,\%$ on the four LIBERO suites, 0.10 percentage points below the official $96.85\,\%$. All 64,761 inferences and 320,142 executed action steps are finite and verified. A separate highest-precision control reaches $97.00\,\%$ and is kept as a control rather than as the main table. Six genuine teaching cases, a physically replayed drawer failure and two clearly separated classes of anomaly (non-finite engineering failures versus normal task failures) are documented, together with all limitations of the claim.

References

[1] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[2] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023.

[3] OpenWAM Authors. An open, modular exploration towards systematic world–action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026.

[4] Wan Team. Wan: Open and advanced large-scale video generative models. *arXiv preprint*, 2025.